

```
class Person {
  #name; #email; #age;

  constructor(name, email, age) {
    this.name = name; // triggers setter
    this.email = email;
    this.age = age;
  }

  set name(v) {
    if (!v || v.trim().length < 2) throw new TypeError('Name too short');
    this.#name = v.trim();
  }

  set email(v) {
    if (!/^[@]+\@[^@]+\.[^@]+$/.test(v)) throw new TypeError('Invalid email');
    this.#email = v.toLowerCase();
  }

  set age(v) {
    if (v < 5 || v > 120) throw new RangeError('Age: 5-120');
    this.#age = v;
  }

  get name() { return this.#name; }
  get email() { return this.#email; }
  get age() { return this.#age; }

  toString() {
    return `[${this.constructor.name}] ${this.name} <${this.email}>`;
  }
}
```

```
}  
}
```

Part 2 — Book Class with Private Fields

Book ka data — title, author, ISBN, availability tracking.

Book — Private State + Factory Method

```
class Book {  
  #available = true;  
  #borrowedBy = null;  
  #borrowCount = 0;  
  
  constructor(title, author, isbn, genre) {  
    this.title = title;  
    this.author = author;  
    this.isbn = isbn;  
    this.genre = genre;  
    this.id = `BK-${isbn.slice(-4)}`;  
  }  
  
  get isAvailable() { return this.#available; }  
  get borrowedBy() { return this.#borrowedBy; }  
  get borrowCount() { return this.#borrowCount; }  
  
  checkout(memberName) {  
    if (!this.#available) throw new Error(`'${this.title}' already borrowed`);  
    this.#available = false;  
    this.#borrowedBy = memberName;  
    this.#borrowCount++;  
    return this;  
  }  
  
  returnBook() {  
    if (this.#available) throw new Error(`'${this.title}' not borrowed`);  
    this.#available = true;  
    this.#borrowedBy = null;  
    return this;  
  }  
  
  toString() {  
    const status = this.#available ? 'Available' : `Borrowed by ${this.#borrowedBy}`;  
    return `[${this.id}] '${this.title}' by ${this.author} - ${status}`;  
  }  
  
  static fromJSON({ title, author, isbn, genre }) {  
    return new Book(title, author, isbn, genre);  
  }  
}
```

Part 3 — Member & Librarian (extends Person)

Member borrow karta hai, Librarian manage karta hai — same Person base.

Member & Librarian — extends Person

```

class Member extends Person {
  #borrowedBooks = [];
  static MAX_BOOKS = 3;
  constructor(name, email, age, memberId) {
    super(name, email, age);
    this.memberId = memberId;
    this.joinedAt = new Date().toISOString().split('T')[0];
  }
  get borrowedBooks() { return [...this.#borrowedBooks]; }
  get canBorrow() { return this.#borrowedBooks.length < Member.MAX_BOOKS; }
  borrowBook(book) {
    if (!this.canBorrow)
      throw new Error(`${this.name} has reached max borrow limit (${Member.MAX_BOOKS})`);
    book.checkout(this.name);
    this.#borrowedBooks.push(book);
    return this;
  }
  returnBook(bookId) {
    const idx = this.#borrowedBooks.findIndex(b => b.id === bookId);
    if (idx === -1) throw new Error('Book not in your borrowed list');
    this.#borrowedBooks[idx].returnBook();
    this.#borrowedBooks.splice(idx, 1);
    return this;
  }
  toString() {
    return `${super.toString()} | ID: ${this.memberId} | Books:
    ${this.#borrowedBooks.length}/${Member.MAX_BOOKS}`;
  }
}

class Librarian extends Person {
  constructor(name, email, age, staffId) {
    super(name, email, age);
    this.staffId = staffId;
    this.role = 'Librarian';
  }
  addBook(library, bookData) {
    const book = Book.fromJSON(bookData);
    library.addBook(book);
    return book;
  }
  removeBook(library, bookId) {
    return library.removeBook(bookId);
  }
  getReport(library) {
    const books = library.getAllBooks();
    const available = books.filter(b => b.isAvailable).length;
    return {
      total: books.length, available,
    }
  }
}

```

```

    borrowed: books.length - available,
    topBooks: books.sort((a,b) => b.borrowCount - a.borrowCount).slice(0,3)
  };
}
}

```

Part 4 — Library Class (Manager)

Sab kuch tie karne wali main Library class — books aur members manage karta hai.

Library — Main Manager Class

```

class Library {
  #books = new Map(); // isbn -> Book
  #members = new Map(); // memberId -> Member
  #records = [];

  constructor(name, city) {
    this.name = name;
    this.city = city;
  }

  addBook(book) {
    this.#books.set(book.id, book);
    return this;
  }

  removeBook(bookId) {
    if (!this.#books.has(bookId)) throw new Error('Book not found');
    this.#books.delete(bookId);
    return this;
  }

  registerMember(member) {
    this.#members.set(member.memberId, member);
    return this;
  }

  getAllBooks() { return [...this.#books.values()]; }
  getAvailable() { return this.getAllBooks().filter(b => b.isAvailable); }
  searchByTitle(q){
    return this.getAllBooks().filter(b =>
    b.title.toLowerCase().includes(q.toLowerCase()));
  }
  searchByGenre(g){
    return this.getAllBooks().filter(b =>
    b.genre.toLowerCase() === g.toLowerCase());
  }

  borrow(memberId, bookId) {
    const member = this.#members.get(memberId);
    const book = this.#books.get(bookId);
    if (!member) throw new Error('Member not found');
    if (!book) throw new Error('Book not found');
    member.borrowBook(book);
    this.#records.push({ memberId, bookId, borrowedAt: new Date(), type:'borrow' });
  }
}

```

```

    return `${book.title} issued to ${member.name}`;
  }

  returnBook(memberId, bookId) {
    const member = this.#members.get(memberId);
    if (!member) throw new Error('Member not found');
    member.returnBook(bookId);
    this.#records.push({ memberId, bookId, returnedAt: new Date(), type:'return' });
    return `Book ${bookId} returned by ${member.name}`;
  }
}

```

Part 5 — Putting It All Together

Complete Integration — Run in Node.js

```

// main.js – Run this in Node.js!
const lib = new Library('Geeta University Library', 'Panipat');

// Create books
const books = [
  new Book('Clean Code', 'Robert Martin', '9780132350884', 'Programming'),
  new Book('You Dont Know JS', 'Kyle Simpson', '9781491904244', 'Programming'),
  new Book('The Pragmatic Programmer', 'Hunt & Thomas', '9780135957059', 'Programming'),
  new Book('Atomic Habits', 'James Clear', '9780735211292', 'Self-Help'),
];
books.forEach(b => lib.addBook(b));

// Create members
const rahul = new Member('Rahul Sharma', 'rahul@geeta.edu', 20, 'M001');
const priya = new Member('Priya Singh', 'priya@geeta.edu', 21, 'M002');
lib.registerMember(rahul).registerMember(priya);

// Create librarian
const librarian = new Librarian('Dr. Meena', 'meena@geeta.edu', 45, 'L001');

// Borrow books
console.log(lib.borrow('M001', 'BK-0884'));
// 'Clean Code' issued to Rahul Sharma
console.log(lib.borrow('M001', 'BK-4244'));
// 'You Dont Know JS' issued to Rahul Sharma
console.log(lib.borrow('M002', 'BK-0884'));
// Error: 'Clean Code' already borrowed

// Search
lib.searchByGenre('Programming').map(b => b.title);
// ['Clean Code', 'You Dont Know JS', 'The Pragmatic Programmer']

// Return
console.log(lib.returnBook('M001', 'BK-0884'));
// Book BK-0884 returned by Rahul Sharma

// Librarian report
console.log(librarian.getReport(lib));
// { total: 4, available: 3, borrowed: 1, topBooks: [...] }

// Add new book via librarian

```

```
librarian.addBook(lib, {  
  title: 'Deep Work', author: 'Cal Newport',  
  isbn: '9781455586691', genre: 'Self-Help'  
});
```

Bonus Challenges

1. FineCalculator mixin add karo — overdue books pe per-day fine calculate karo.
2. EBook class banao (extends Book) — extra: fileSize, format (PDF/EPUB), downloadUrl.
3. Library.searchByAuthor(name) method add karo.
4. Member.borrowHistory array mein returned books bhi track karo.
5. Library ko JSON export/import karne ki functionality add karo — static fromJSON.